Laboratorio 2: Diseño responsive y animaciones CSS

Continuación del proyecto Portal Académico Universitario · Aplicaciones Web · Escuela Politécnica Nacional

Duración: 90 minutos | Nivel: intermedio | Modalidad: práctica individual

Objetivos de aprendizaje

• Explicar responsive web design y el enfoque mobile-first.

• Aplicar unidades relativas, Flexbox, CSS Grid y media queries.

• Adaptar menús, tarjetas, tablas y formularios a distintos dispositivos.

• Diferenciar transition, transform, animation y @keyframes.

• Aplicar accesibilidad mediante prefers-reduced-motion.

Plan de sesión

Bloque	Actividad	Tiempo
1	Diagnóstico y fundamentos responsive	10 min
2	CSS Grid, Flexbox y media queries	30 min
3	Transiciones y animaciones CSS	30 min
4	Reto integrador y comprobación	20 min

Parte I. Fundamentos del diseño adaptable (10 min)

1. ¿Qué es responsive web design?

Es una técnica para reorganizar la interfaz según el espacio disponible. En escritorio se muestran varias tarjetas por fila; en móvil se apilan. Conceptos: viewport (área visible), breakpoint (punto de cambio), media query (condición CSS), mobile-first (diseñar primero para pantallas pequeñas).

Diagnóstico (3 min): abrir index.html, pulsar F12, activar la vista de dispositivo y probar anchos 375, 768 y 1440 px. Observar menú, tarjetas y desplazamiento horizontal.

Parte II. Adaptabilidad del portal (30 min)

2. Preparar el proyecto

```
portal-academico/
├── index.html
├── css/estilos.css
├── img/universidad.jpg
└── README.md
```

Confirmar en <head> que estén presentes:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<link rel="stylesheet" href="css/estilos.css">
```

3. Unidades CSS

```
width: 300px;       /* fija */
width: 80%;        /* relativa al contenedor */
padding: 2rem;     /* relativa a fuente raíz */
width: 80vw;       /* relativa al viewport */
max-width: 1100px; /* límite */
```

Actividad: reducir el ancho del navegador y explicar cuándo conviene cada unidad.

En CSS existen unidades fijas y relativas que permiten controlar el tamaño de los elementos de una página web. En tu proyecto **Portal Académico Universitario**, estas unidades ayudan a que el diseño se adapte a celulares, tabletas y computadoras.

¿Qué significa cada unidad?

Código CSS	**Significado**	**¿Cuándo conviene usarla?**
width: 300px;	Ancho fijo de 300 píxeles CSS.	Para elementos que necesitan una dimensión concreta, como un icono o un botón con tamaño controlado.
width: 80%;	Ocupa el 80 % del ancho de su contenedor.	Para imágenes, formularios o secciones que deben adaptarse al espacio disponible.
padding: 2rem;	Espacio interior equivalente a dos veces el tamaño de fuente raíz.	Para definir márgenes interiores que escalen según el tamaño de texto configurado en el documento.
width: 80vw;	Ocupa el 80 % del ancho del viewport (área visible del navegador).	Para secciones que deben dimensionarse respecto a la pantalla, independientemente del ancho del contenedor padre.
max-width: 1100px;	Limita el ancho máximo a 1100 píxeles CSS.	Para evitar que el contenido se extienda demasiado en monitores grandes.

4. Cuadrícula responsive con Grid (10 min)

Sustituir la definición anterior de .contenedor-cursos por:

```
.contenedor-cursos {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(min(100%, 250px), 1fr));
    gap: 20px;
    width: 100%;
}
```

auto-fit ajusta el número de columnas; minmax establece límites; min(100%, 250px) evita desbordamiento en pantallas estrechas; 1fr distribuye el espacio disponible.

5. Menú adaptable con Flexbox (7 min)

```
.navegacion ul {
    display: flex;
    justify-content: center;
    flex-wrap: wrap;
    gap: 12px 25px;
    list-style: none;
    padding: 0;
    margin: 0;
}
.navegacion a {
    display: inline-block;
    padding: 10px 12px;
    color: white;
    text-decoration: none;
    border-radius: 6px;
}
```

flex-wrap permite que los enlaces se reorganicen cuando el espacio sea insuficiente.

6. Media queries (8 min)

```
@media (min-width: 1025px) {
    .encabezado h1 { font-size: 2.2rem; }
}
@media (min-width: 601px) and (max-width: 1024px) {
    main { padding: 20px; }
    .encabezado h1 { font-size: 1.8rem; }
}
@media (max-width: 600px) {
    .encabezado { padding: 25px 12px; }
    .encabezado h1 { font-size: 1.4rem; }
    .navegacion ul { flex-direction: column; align-items: stretch; gap: 5px; }
    .navegacion a { display: block; text-align: center; }
    main { padding: 12px; }
    .bienvenida, .contacto, .noticias { padding: 18px; }
}
```

Revisar si existen reglas @media anteriores que afecten a los mismos selectores y consolidarlas cuando sea necesario.

7. Imágenes, formularios y tablas (5 min)

```
img { max-width: 100%; height: auto; }
input, textarea, button { max-width: 100%; }
.tabla-responsive { width: 100%; overflow-x: auto; }
.tabla-responsive table { width: 100%; min-width: 600px; border-collapse: collapse; }
<div class="tabla-responsive" role="region" aria-label="Horario académico" tabindex="0">
```

```
    <table>
        <caption>Horario académico semanal</caption>
        <!-- Conservar las filas de la tabla existente -->
    </table>
</div>
```

Parte III. Introducción a las animaciones CSS (30 min)

8. Conceptos

Propiedad	Función
transition	Suaviza cambios entre estados
transform	Mueve, rota o escala visualmente
animation	Ejecuta una secuencia
@keyframes	Define estados de la secuencia

9. Botón con transición (5 min)

```
.contacto button {
    background-color: #2563eb; color: white;
    padding: 12px 24px; border: none; border-radius: 8px;
    cursor: pointer;
    transition: background-color .3s ease, transform .3s ease;
}
.contacto button:hover {
    background-color: #1e40af;
    transform: scale(1.05);
}
```

10. Transformaciones (5 min)

```
transform: translateX(30px);
transform: translateY(-10px);
transform: rotate(20deg);
transform: scale(1.2);
```

Actividad: aplicar cada transformación a .tarjeta:hover y observar el efecto.

11. Círculo animado con @keyframes (10 min)

Añadir dentro de <main> una nueva sección:

```
<section class="laboratorio-animacion">
    <h2>Laboratorio de animaciones</h2>
    <p>Ejemplo de movimiento con CSS.</p>
    <div class="escenario"><div class="pelota"></div></div>
</section>
```

Agregar al archivo CSS:

```
.laboratorio-animacion {
    padding: 25px; background: white; border-radius: 12px; margin-bottom: 30px;
}
.escenario {
    height: 120px; background: #eff6ff; border: 2px solid #bfdbfe;
    border-radius: 12px; overflow: hidden; position: relative;
}
.pelota {
    width: 50px; height: 50px; border-radius: 50%;
    background: #2563eb; position: absolute; top: 35px; left: 0;
```

```
    animation: moverPelota 3s ease-in-out infinite alternate;
}
@keyframes moverPelota {
    0% { left: 0; background: #2563eb; }
    50% { background: #f59e0b; }
    100% { left: calc(100% - 50px); background: #dc2626; }
}
```

Explicar: 0% es el inicio; 50% cambia a naranja; 100% alcanza el borde derecho y se vuelve rojo. infinite repite y alternate invierte el sentido. Probar duraciones de 1s, 3s y 8s.

12. Entrada escalonada de tarjetas (5 min)

```
@keyframes entradaTarjeta {
    from { opacity: 0; transform: translateY(25px); }
    to { opacity: 1; transform: translateY(0); }
}
.tarjeta { animation: entradaTarjeta .7s ease-out both; }
.tarjeta:nth-child(2) { animation-delay: .15s; }
.tarjeta:nth-child(3) { animation-delay: .3s; }
.tarjeta:nth-child(4) { animation-delay: .45s; }
```

13. Accesibilidad (5 min)

```
@media (prefers-reduced-motion: reduce) {
    *, *::before, *::after {
        animation-duration: 0.01ms !important;
        animation-iteration-count: 1 !important;
        transition-duration: 0.01ms !important;
        scroll-behavior: auto !important;
    }
}
```

Esta regla respeta la preferencia del usuario por reducir movimiento.

Parte IV. Reto integrador (20 min)

Reto: Portal Académico Responsive Plus

☐ Asegurar que las tarjetas se adapten a escritorio, tableta y móvil.

☐ Convertir el menú en una disposición vertical para teléfonos.

☐ Crear una figura geométrica que se desplace y cambie de color con @keyframes.

☐ Agregar un efecto hover a una tarjeta o botón.

☐ Incorporar prefers-reduced-motion.

Reto adicional: lograr que el círculo recorra los cuatro bordes de un contenedor mediante 0%, 25%, 50%, 75% y 100%.

Rúbrica de evaluación

Criterio	Puntaje
Diseño adaptable en tres tamaños	3

Navegación responsive	2
Animación con @keyframes	2
Efecto hover o transición	1
Código y accesibilidad	2
TOTAL	10

Entrega y comprobación

Entregar un ZIP con index.html, css/estilos.css, imágenes y dos capturas: escritorio y móvil. Verificar enlaces, formulario, tarjetas, tabla, animación y ausencia de desbordamientos horizontales no deseados.

Preguntas de cierre

• ¿Qué diferencia existe entre media queries y Grid con auto-fit?

Son complementarias, no excluyentes. Las media queries reaccionan al ancho de la ventana/pantalla (@media (max-width: 600px)) y sirven para cambiar reglas concretas: apilar el menú, reducir paddings, cambiar tamaños de fuente. Grid con auto-fit reacciona al ancho del contenedor: con minmax(min(100%, 250px), 1fr) calcula solo cuántas columnas de 250px caben y las distribuye, sin escribir un solo breakpoint. En la práctica: Grid con auto-fit resuelve el caso más común (reordenar tarjetas), y las media queries atienden los ajustes finos que Grid no puede expresar.

• ¿Por qué conviene usar unidades relativas?

Porque se adaptan al contexto y hacen el diseño verdaderamente escalable:

rem / em → relativas a la fuente (raíz o padre).

% → relativa al contenedor.

vw / vh → relativas al viewport.

Con unidades relativas una misma regla funciona en cualquier pantalla (móvil, tableta, 4K) sin duplicar estilos. Las fijas (px) quedan reservadas para detalles que no deben cambiar, como bordes finos. max-width: 1100px es la excepción que pone un *límite* y evita que el contenido se estire de más en monitores grandes.

• ¿Cuándo utilizar transition y cuándo @keyframes?

transition: Suaviza un cambio entre dos estados que dispara una acción (un :hover o :focus). Para el botón de contacto o el efecto en las tarjetas.

@keyframes + animation: Ejecuta una secuencia con varios estados o que corre sola y en bucle (infinite, alternate). Ideal para el círculo que cruza la escena o la entrada escalonada de tarjetas.

Regla: Para un simple ida y vuelta por interacción usar transition; para una secuencia de varios pasos o animación continua usa @keyframes.

• ¿Cómo afecta una animación excesiva a la experiencia del usuario?

Cansa la vista y distrae del contenido (sobre todo la opacidad constante y el movimiento).

Ralentiza el dispositivo, sobre todo en móviles, al consumir GPU y memoria.

Puede provocar malestar como mareo o náuseas y hasta crisis en personas con epilepsia fotosensible o sensibilidad al movimiento.

Por eso se estilizan cortas, sutiles y se respeta prefers-reduced-motion, que reduce animaciones cuando el usuario lo pide en el sistema.

Conclusión: una interfaz profesional debe ser adaptable, accesible y funcional. Este laboratorio prepara la transición hacia JavaScript y la manipulación del DOM.